

Control Structures

L17 - Selection Statements 1

Department of Computer Science
University of Pretoria

18 April 2024

- **Selection statements** (aka decision statements) are used to control the flow of a program based on certain conditions.
- The selection statements **only** allow specific commands to be executed depending on the outcome of the condition.

- The general principle of a selection statement is that specific statements will execute if the condition is **true**, otherwise they will not execute.
- There are three selection statements defined in C++, namely: (i) **Ternary-IF statement**, (ii) **IF-statement** and the (iii) **SWITCH-statement**.

- The **ternary** operator, also known as the conditional operator, is a concise way to express a simple conditional statement in C++.
- It takes three operands: (i) a **condition**, (ii) a result for **true**, and a (iii) result for **false**.

The syntax is as follows:

```
<condition> ? <expression_true> : <expression_false>;
```

- **<condition>:** Must be a boolean expression
 - Direct boolean values (true, false)
 - Comparisons (a == b, a < b, a >= b, etc.)
 - Logical operations (a && b, a || b, !a, etc.)

`<expression_true>` **and**

`<expression_false>`: Any expressions
that results in a value :

- Literals (42, 3.14, 'c', etc.)
- Variables (myVar, result, etc.)
- Function calls (calculate(),
getMax(a, b), etc.)
- Arithmetic expressions ($a + b$, $a * b$,
etc.)

- **Both** the `<expression_true>` and `<expression_false>` need to be **of the same type**, or convertible to the same type.

If the `<condition>` is true, the operator evaluates and returns the result of evaluating `<expression_true>`; otherwise, it returns the resultant value after the evaluation of `<expression_false>` has taken place.

Simple Ternary If-Statements

In C++ the ternary operator's structure, maps onto a simple if-statement and is therefore referred to as a ternary-if.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int a, b;
6     cout << "Enter two numbers:\n";
7     cin >> a >> b;
8     int max = (a > b) ? a : b;
9     cout << "The greater number is -" << max << endl;
10    return 0;
11 }
```

In this example, the user is prompted to enter two numbers. The ternary operator is used to determine the greater of the two numbers and assign it to the variable `max`.

The ternary operator can also use function calls as the expressions for both the true and false cases as shown below. What will be the output if user enters 10 and 20 respectively for either operation?

```
1 #include <iostream>
2 using namespace std;
3 int add(int a, int b) {
4     return a + b; }
5 int subtract(int a, int b) {
6     return a - b;
7 }
8 int main() {
9     int num1, num2;
10    char operation;
11    cout << "Enter two numbers: -";
12    cin >> num1 >> num2;
13    cout << "Choose operation - (a for add, -s for subtract): -";
14    cin >> operation;
15    bool condition = (operation == 'a');
16    int result = condition ?
17        add(num1, num2) : subtract(num1, num2);
18    cout << "The result is: -" << result << endl;
19    return 0;
20 }
```

The ternary operator can be nested IN C++, This means that the expression to be evaluated can itself be another ternary if-statement.

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     int score;
5     cout << "Enter your score: -";
6     cin >> score;
7     //Simple nesting
8     string grade = (score < 50 ) ? " Fail" :
9                    (score < 75) ? " Pass" : " Distinction";
10    cout << "Your grade is:-" << grade << endl;
11    //Complex nesting
12    string detailed = (score >= 90)? "High-Distinction" :
13                     (score >= 75) ? "Distinction" :
14                     (score >= 60) ? "Credit" :
15                     (score >= 50) ? "Pass" : "Fail";
16    cout << "Your detailed grade is:-" << detailed << endl;
17    return 0;
18 }
```

What output is produced when the user inputs 74?

Introduction to IF Statements

- An if-statement evaluates a condition. Depending on the outcome of the condition, either `true` or `false`, a single statement or block of statements are executed.
- It allows you to execute a block of code only if a specified condition is **true**. You can provide an alternative block of code to execute if the condition is false using the **else** clause.

Simple IF Statements

The basic syntax of the if-statement is as follows:

C++ Single Statement	C++ Block of Statements
if (<condition>) <statement>;	if (<condition>) { <block>; }
if (<condition>) <true_statement>; else <false_statement>;	if (<condition>) { <true_block>; } else { <false_block>; }

The simple if-statement translates directly to the ternary if-statement that includes both a `<true_statement>` and a `<false_statement>`. That is the high-level code of:

```
if (<condition>)  
    <true_statement>;  
else  
    <false_statement>;
```

translates to the ternary if-statement of:

```
<condition> ? <true_statement> : <false_statement>;
```

Consider the following C++ program illustrating the use of a single statement for the result of the condition.

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     int x;
5     cout << "Type-in-an-integer-value:-";
6     cin >> x;
7     if (x < 10)
8         cout << "The-value-is-<-10" << endl;
9     else
10        cout << "The-value-is->=10" << endl;
11    return 0;
12 }
```

In this C++ program, the value of x is compared to 10, and a message is displayed based on whether x is less than 10 or greater than or equal to 10.

The following C++ program uses blocks of code (encapsulated in the curly brackets). This C++ program takes two integers from the user, compares them, and either adds them or subtracts them based on their values. It then displays the results.

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      int x, y;
5      cout << "Type-in-an-integer-value:-";
6      cin >> x;
7      cout << "Type-in-another-integer-value:-";
8      cin >> y;
9      if (x < y) {
10         cout << x << "-is-less-than-" << y << endl;
11         cout << x << "-plus-" << y << "-=" << x + y << endl;
12     } else {
13         cout << x << "-is-not-less-than-" << y << endl;
14         cout << x << "-minus-" << y << "-=" << x - y << endl;
15     }
16     return 0;
17 }
```

Note that the if-statement of the previous example could also be rewritten in the following way using two different functions (provided they are appropriately defined with the correct statements).

```
if (x < y) {  
    printLessThanMessage();  
} else {  
    printGreaterEqualMessage();  
}
```


Write a C++ program using a ternary operator to check whether a student passed the semester test or not. If the student got 50% and above, print, "You passed the test". If the student got less than 50%, print "Sorry, you failed the test". The program must accept input from the user.

Exercise Solution:

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     // Variable to store the test mark and result
5     int mark, result;
6     // Prompt the user to enter the semester test mark
7     cout << "Enter-your-semester-test-mark:-";
8     cin >> mark;
9
10    // Check if the score is 50% or above using the ternary operator
11    cout << (mark >= 50.0 ? "You-passed-the-test!" :
12    "Sorry, -you-failed-the-test.");
13    return 0;
14 }
```

As homework, convert the above code to an if-statement.

Write a C++ program with a function called *string classifyDegree(int degree)* using ternary operators that takes a number as input and returns whether the student is studying for a BSc, BEng or BCom. The degree categories are defined as follows: **BSc:** 1 - 3, **BEng:** 4 - 6, **BCom:** 7 - 8.

Homework: Once you are done, write another C++ program which converts the function to use nested if statements instead.

Exercise Solution: classifyDegree() Function

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 // Function to classify the degree based on the input number
5 string classifyDegree(int degree) {
6     // Variable to store the degree category
7     string category;
8     // Using ternary operators to classify the degree
9     category = (degree >= 1 && degree <= 3) ? "BSc" :
10                (degree >= 4 && degree <= 6) ? "BEng" :
11                (degree >= 7 && degree <= 8) ? "BCom" :
12                "Unknown-Degree"; // Default case
13     return category;
14 }
```

Exercise Solution: Main Function

```
1  int main() {  
2      // Variable to store the degree  
3      int degree;  
4      // Prompt the user to enter the degree number  
5      cout << "Enter the degree number:-";  
6      cin >> degree;  
7      // Call the classifyDegree function and store the result  
8      string degreeCategory = classifyDegree(degree);  
9      // Print the degree category  
10     cout << "Degree category:-" << degreeCategory << endl;  
11     return 0;  
12 }
```